Web Information Retrieval

Ingo Frommholz

University of Wolverhampton

March 6, 2022

Table of Contents

Some Literature I

Information Retrieval

- Ricardo Baeza-Yates and Berthier Ribero-Neto. *Modern Information Retrieval 2nd Edition*. Addison Wesley, 2011. ISBN 978-0-321-41691-9
- Christopher D. Manning, Prabhakar Raghavan and Hinrich Schütze, *Introduction to Information Retrieval*, Cambridge University Press. 2008. Available online at `http://nlp.stanford.edu/IR-book/`
- Further IR resources: `http://nlp.stanford.edu/IR-book/information-retrieval.html`

Some Literature II

Web Information Retrieval Papers

- S. Brin and L. Page, *The anatomy of a large-scale hypertextual web search engine*, Proceedings of the 7th International World-Wide Web Conference (WWW 1998), pages 107–117, Brisbane, Australia, 1998. `http://infolab.stanford.edu/~backrub/google.html`
- L. Page, S. Brin, R. Motwani, and T. Winograd, *The PageRank Citation Ranking: Bringing Order to the Web*. Technical Report, 1998. `http://ilpubs.stanford.edu:8090/422/`
- J. Kleinberg, *Authoritative sources in a hyperlinked environment*, Proceedings of the 9th Annual ACM-SIAM Symposium on Discrete Algorithms, 1998, pp. 668-677. `http://www.cs.cornell.edu/home/kleinber/auth.pdf`

Seeking Information on the Web

The World Wide Web

Technical Definition:

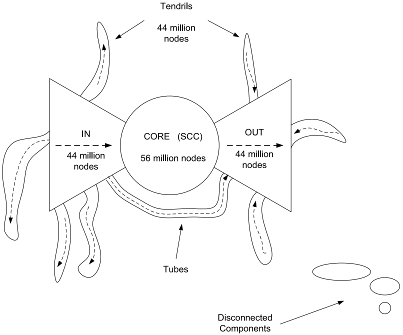
> *"All Internet resources and users using the Hypertext Transfer Protocol (HTTP)"*

More general definition (Sir Tim Berners-Lee):

> *"The World Wide Web is the universe of network-accessible information, an embodiment of human knowledge"*

Web Bowtie Structure

[Broder et al., 2000]



- CORE: Strongly Connected Components (SCC)
- IN: pages that can reach SCC, but cannot be reached from it
- OUT: pages accessible from SCC, but not linking back to it
- Tendrils: pages that cannot reach SCC and cannot be reached from SCC
- Tubes: Pages linking from IN to OUT bypassing SCC

Trying to find what you want on the Web...

Web Search Taxonomy

[Broder, 2002, Rose and Levinson, 2004]

Navigational Aim is to reach a particular site

Informational Acquire some information present on a web site

Transactional Find web pages where some further interaction will happen (e.g., shopping)

Resource To get access to some online resource

Problems Seeking Information on the Web

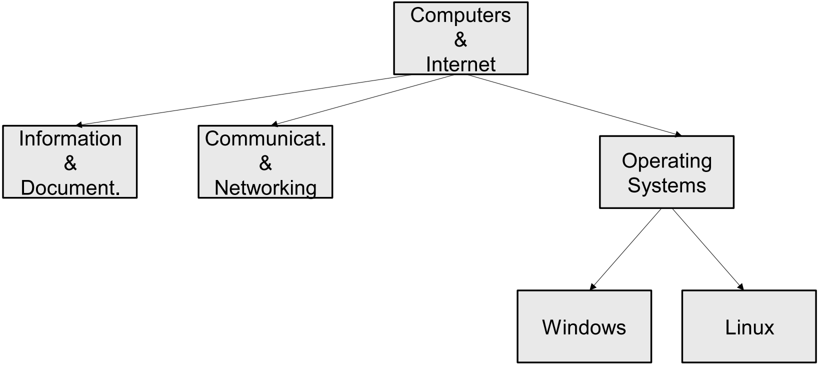
- Exponental information growth
- Vanishing documents (404)
- Heterogeneous document types
- Document quality
- Multilinguality

Browsing

- Document search can be simplified by navigating to interesting/relevant categories
- Catalogues like (the now defunct) *Yahoo!* and DMOZ offered a **hierarchical category scheme**
- Each Web document in this catalogue is assigned to one or more categories, which gives useful hints about the document topic
- Topic gets more specific the deeper we get

Hierarchical Category Scheme Example

Yahoo! Computers & Internet



Hierarchical Category Scheme Example

DMOZ

Browsing Advantages and Disadvantages

+ Possible higher precision as search space gets rapidly confined

Browsing Advantages and Disadvantages

- + Possible higher precision as search space gets rapidly confined
- + Handling of homonyms and polysemes:

Browsing Advantages and Disadvantages

+ Possible higher precision as search space gets rapidly confined
+ Handling of homonyms and polysemes:
  - Search for `tree` in a category "Forestry" filters out all documents about the tree data structure

Browsing Advantages and Disadvantages

- + Possible higher precision as search space gets rapidly confined
- + Handling of homonyms and polysemes:
  - Search for `tree` in a category "Forestry" filters out all documents about the tree data structure
- + More relevant documents can be found by investigating other documents in the same category

Browsing Advantages and Disadvantages

- \+ Possible higher precision as search space gets rapidly confined
- \+ Handling of homonyms and polysemes:
  - Search for `tree` in a category "Forestry" filters out all documents about the tree data structure
- \+ More relevant documents can be found by investigating other documents in the same category
- \+ User does not have to formulate a query

Browsing Advantages and Disadvantages

- + Possible higher precision as search space gets rapidly confined
- + Handling of homonyms and polysemes:
  - Search for `tree` in a category “Forestry” filters out all documents about the tree data structure
- + More relevant documents can be found by investigating other documents in the same category
- + User does not have to formulate a query
- + No deep knowledge of the subject area necessary, but:

Browsing Advantages and Disadvantages

+ Possible higher precision as search space gets rapidly confined
+ Handling of homonyms and polysemes:
    - Search for `tree` in a category “Forestry” filters out all documents about the tree data structure
+ More relevant documents can be found by investigating other documents in the same category
+ User does not have to formulate a query
+ No deep knowledge of the subject area necessary, but:
– User needs to know beforehand which category to visit ($\rightarrow$ Search)

Browsing Advantages and Disadvantages

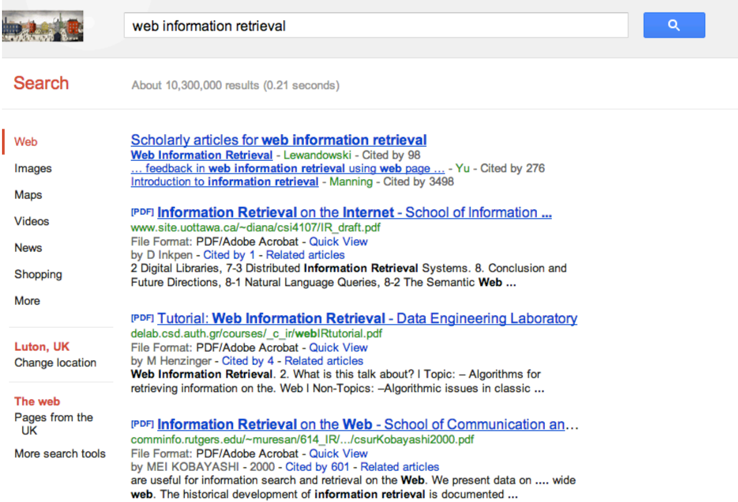
- + Possible higher precision as search space gets rapidly confined
- + Handling of homonyms and polysemes:
  - Search for `tree` in a category “Forestry” filters out all documents about the tree data structure
- + More relevant documents can be found by investigating other documents in the same category
- + User does not have to formulate a query
- + No deep knowledge of the subject area necessary, but:
- – User needs to know beforehand which category to visit ($\rightarrow$ Search)
- – Many documents uncategorised

Searching

- The vast amount of Web documents cannot be made available without search technology
- "The Web is a jungle" (Sergey Brin)
- Often there is an interplay between searching and browsing
- We can apply standard IR methods (e.g. VSM, inverted lists, $tf \times idf$) plus **link analysis**

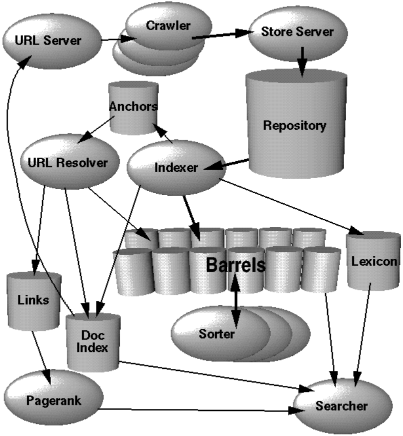
Anatomy of a Web Search Engine: The early Google

Example: Google



Google High Level Architecture

[Brin and Page, 1998]



Web Search Engine Components

Webcrawler/Spider Collects Web sites, interacts with Web servers, follows links to new Web pages

Web Search Engine Components

Webcrawler/Spider Collects Web sites, interacts with Web servers, follows links to new Web pages

Parser/Indexer Extracts keywords from full texts, indexes the documents

Web Search Engine Components

Webcrawler/Spider Collects Web sites, interacts with Web servers, follows links to new Web pages

Parser/Indexer Extracts keywords from full texts, indexes the documents

Storage System Efficient storage of the extracted and processed information (e.g. in a database with inverted lists)

Web Search Engine Components

Webcrawler/Spider Collects Web sites, interacts with Web servers, follows links to new Web pages

Parser/Indexer Extracts keywords from full texts, indexes the documents

Storage System Efficient storage of the extracted and processed information (e.g. in a database with inverted lists)

User Interface Query input, interacts with the storage system

Web Search Engine Components

Webcrawler/Spider Collects Web sites, interacts with Web servers, follows links to new Web pages

Parser/Indexer Extracts keywords from full texts, indexes the documents

Storage System Efficient storage of the extracted and processed information (e.g. in a database with inverted lists)

User Interface Query input, interacts with the storage system

- Rather simple components

Web Search Engine Components

Webcrawler/Spider Collects Web sites, interacts with Web servers, follows links to new Web pages

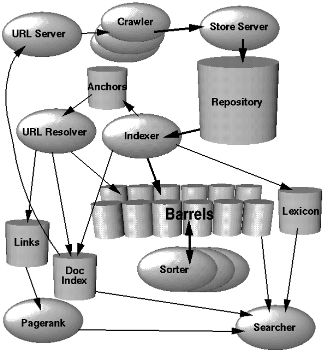
Parser/Indexer Extracts keywords from full texts, indexes the documents

Storage System Efficient storage of the extracted and processed information (e.g. in a database with inverted lists)

User Interface Query input, interacts with the storage system

- Rather simple components
- Has to cope with huge amounts of data and very high access rates

Google Architecture: Components

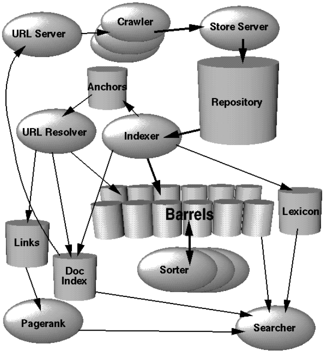


URL Server Collects lists of URLs to visit from the **document index**, sends them to the crawler

Crawler Collects and accesses Web sites

Store Server Compresses Web sites, stores them in a **repository**, assigns document IDs

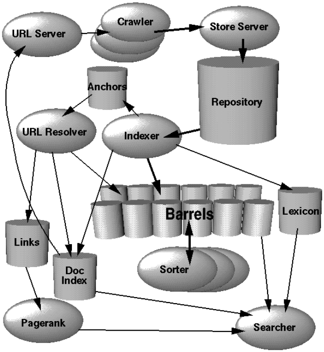
Google Architecture: Components



Indexer Reads the repository, decompresses and parses the data; converts the documents into an enriched set of terms (called **Hits**)

- Hits can contain the word, its position in the document, the font size and capitalisation
- Hits are sorted into **barrels** ⇝ partially sorted forward index and inverted index

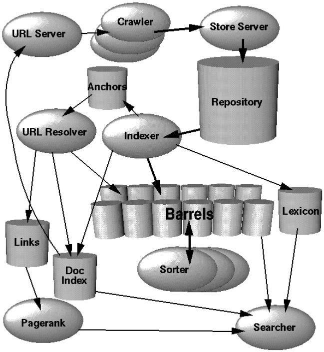
Google Architecture: Components



Indexer (cont'd) Parses all links in the document and stores their important information (source destination and text) in the **anchors** file. Also maintains the **lexicon** of all words (represented as word IDs)

URL Resolver Reads the anchors file, converts relative into absolute URLs and document IDs, generates the **links** database

Google Architecture: Components

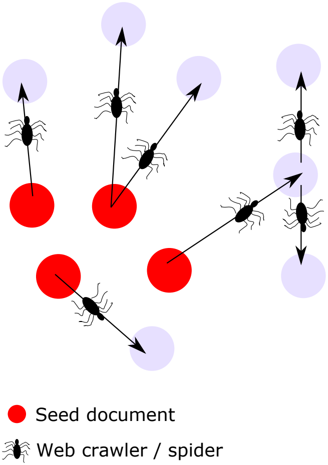


Sorter Generates inverted lists from the barrels, stores them again in barrels

Page Rank Computes the **PageRank** from the link structure in the links database

Searcher Processes search requests; uses PageRank, inverted lists and the lexicon to answer queries

Web Crawling



- **Crawler:** Software for downloading Web pages
- Collect (unknown) Web pages, build the repository
- Analyse and monitor Web pages
- Starting from a set of **seed documents**, following links
- Simple example: Unix `wget`
- Apache Nutch

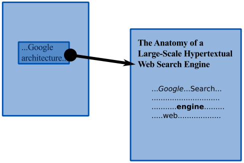
Google Hit Types

Google not only looks for the occurrence of a term (or word), but also separates between 3 different types of occurrences (called **Hits**):

Fancy Hit Includes hits occurring in a URL, title, or meta tag

Anchor Hit Special kind of fancy hit, occurrence in an anchor text linking to the page (anchor propagation)

Plain Hit All the rest



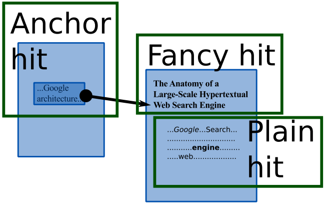
Google Hit Types

Google not only looks for the occurrence of a term (or word), but also separates between 3 different types of occurrences (called **Hits**):

Fancy Hit Includes hits occurring in a URL, title, or meta tag

Anchor Hit Special kind of fancy hit, occurrence in an anchor text linking to the page (anchor propagation)

Plain Hit All the rest



Ranking Function I

- Google's architecture enables the efficient calculation of important figures/signals needed to rank documents w.r.t. the query
  - Precise ranking function kept secret
- Ranking function considers the position of terms in the document, their font size and capitalisation information

Ranking Function II

- Using the inverted index, it retrieves all hits of a query term in a document (full text, fancy and anchor)
- Doesn't use *tf* $\times$ *idf*, but based on the hits computes the dot product between a type-weight vector (full text, anchor, fancy) and a count-weight vector (counting number of full text, anchor and fancy hits) to create the document score (RSV)
- The RSV can be combined with the PageRank

Link Analysis with PageRank and HITS

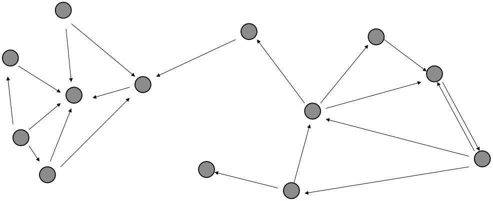
Web Information Retrieval

- Search engines apply known IR techniques (like inverted lists, $tf \times idf$)
- The approaches so far were just content-based (computing the retrieval status value (RSV) w.r.t. a query, rank documents w.r.t. decreasing RSV)
- On the Web we have another source of information: the **link structure**
  - As we've seen, Google stores more than an inverted index, but also information about the link structure
- Performing a **link analysis**, we can extract from the link structure some further information going beyond mere document content

The Web as a Hypertext

Definition (Hypertext)

A **Hypertext** is a set of vertices (nodes) and directed edges (links) forming a graph. The Web can be regarded as a cyclic graph.



Hypertext and Web Information Retrieval

- **Hypertext IR** uses the node content and the link structure for Information Retrieval
- In that sense Web IR is a special case of Hypertext IR
  - In fact, Hypertext IR has a longer history than Web IR
- Possibilities include:
  - Identify the “importance” of documents and of hubs and authorities (**PageRank**, **HITS**)

PageRank I

[Page et al., 1998]

- Famous Google approach
- Determines the **authority** of a document (or page)
- **Assumption**: a document is more authoritative, the more other documents link to it
- *Iterative* calculation of the PageRank by propagating the PageRank to successor nodes

PageRank II

[Page et al., 1998]

- Simple ranking scheme: Use query q to filter documents (like in Boolean retrieval) and then rank matching documents according to descending PageRank ($PR(d)$)
- Elaborated ranking scheme: Linear combination of different relevance signals, e.g. RSV of a document d w.r.t. query q using $tf \times idf$ and the vector space model:

$$RSV^*(d,q) = \alpha\ RSV(d,q) + (1-\alpha)\ PR(d)$$

Random Surfer Model

The assumption behind Page Rank is that of a **random surfer**.

Random Surfer Model

- Random walks on the Web graph
- "Random surfer" clicks on successive links at random $\rightsquigarrow$ equal probability of each link to be clicked
- Surfer gets bored and periodically jumps to another random page

PageRank Calculation I

Iterative Computation of PageRank $PR(p)$ for a page p

$$PR(p) = \overbrace{(1-\beta) \times \frac{1}{N}}^{\text{Surfer gets bored}} + \underbrace{\beta \times \sum_{q \rightarrow p} \frac{PR(q)}{out(q)}}_{\text{Randomly following links}}$$

N:	Number of nodes / pages
β:	Constant between 0 and 1
$out(q)$:	Number of outgoing links from page q
$q \rightarrow p$:	There is a link from q to p

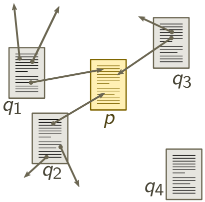
PageRank Calculation II

- PageRank formula is applied iteratively
- Initially, each page has a PageRank of 1

PageRank and the Random Surfer Model

The PageRank formula reflects the Random Surfer Model

$$PR(p) = \overbrace{(1-\beta) \times \frac{1}{N}}^{\text{Surfer gets bored}} + \underbrace{\beta \times \sum_{q \rightarrow p} \frac{PR(q)}{out(q)}}_{\text{Randomly following links}}$$

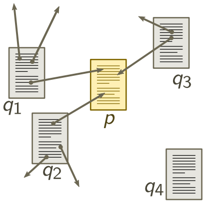


- User randomly jumps to an arbitrary Web page with probability $(1-\beta)$; each of these Web pages (including p) has probability $1/N$, so $(1-\beta) \times 1/N$ is the probability that the user reaches p by an arbitrary random jump (left part of the formula)

PageRank and the Random Surfer Model

The PageRank formula reflects the Random Surfer Model

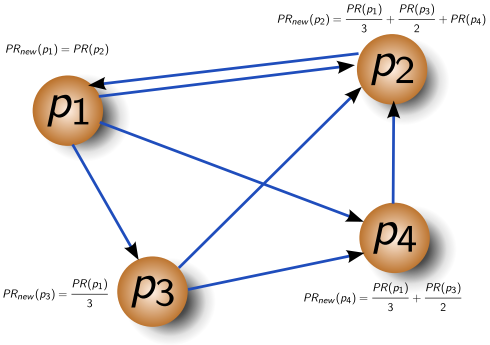
$$PR(p) = \overbrace{(1-\beta) \times \frac{1}{N}}^{\text{Surfer gets bored}} + \underbrace{\beta \times \sum_{q \to p} \frac{PR(q)}{out(q)}}_{\text{Randomly following links}}$$



- β is the probability that the user randomly follows a link to reach another page. $\frac{PR(q)}{out(q)}$ is the probability that the user reaches p from another page q linking to p. If we take all pages q linking to p into account, we get $\sum_{q \to p} \frac{PR(q)}{out(q)}$

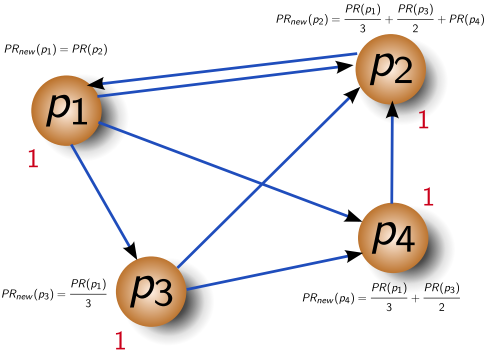
PageRank Example

Example (set $\beta = 1$): $PR(p) = \sum_{q \to p} \frac{PR(q)}{out(q)}$



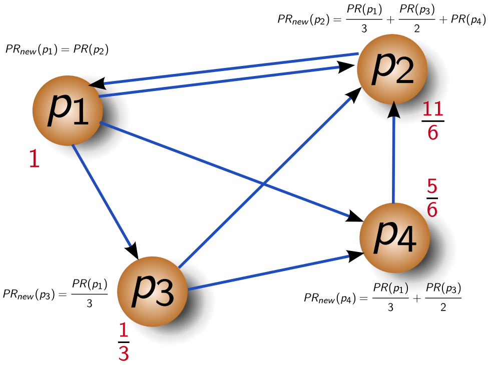
PageRank Example

Example (set $\beta = 1$): $PR(p) = \sum_{q \to p} \frac{PR(q)}{out(q)}$



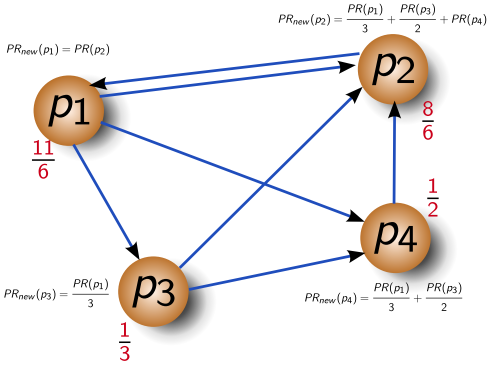
PageRank Example

Example (set $\beta = 1$): $PR(p) = \sum_{q \rightarrow p} \frac{PR(q)}{out(q)}$



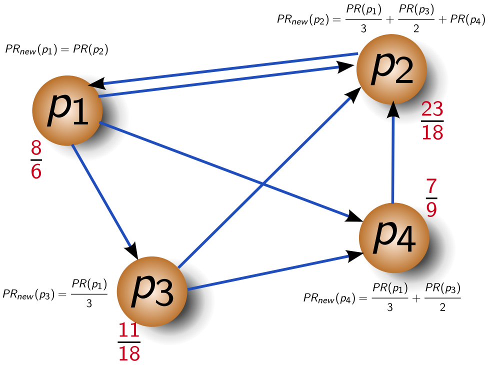
PageRank Example

Example (set $\beta = 1$): $PR(p) = \sum_{q \to p} \frac{PR(q)}{out(q)}$



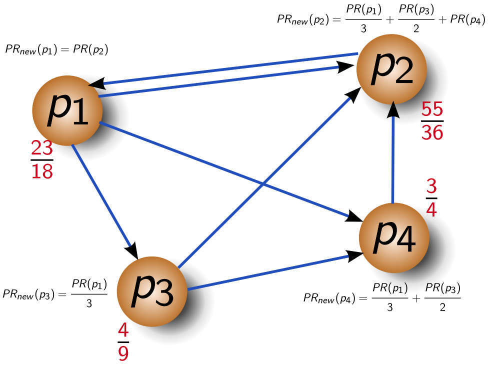
PageRank Example

Example (set $\beta = 1$): $PR(p) = \sum_{q \to p} \frac{PR(q)}{out(q)}$



PageRank Example

Example (set $\beta = 1$): $PR(p) = \sum_{q \to p} \frac{PR(q)}{out(q)}$



Page Rank – Discussion

+ Offline computation independently from the query – this allows for computing the PageRank for billions and more Web pages!

Page Rank – Discussion

- + Offline computation independently from the query – this allows for computing the PageRank for billions and more Web pages!
- + PageRank emphasises popular pages

Page Rank – Discussion

- + Offline computation independently from the query – this allows for computing the PageRank for billions and more Web pages!
- + PageRank emphasises popular pages
- + Good results for home page search

Page Rank – Discussion

- \+ Offline computation independently from the query – this allows for computing the PageRank for billions and more Web pages!
- \+ PageRank emphasises popular pages
- \+ Good results for home page search
- +- Page Rank prefers entry pages of Web sites

Page Rank – Discussion

- \+ Offline computation independently from the query – this allows for computing the PageRank for billions and more Web pages!
- \+ PageRank emphasises popular pages
- \+ Good results for home page search
- +- Page Rank prefers entry pages of Web sites
- – Other methods have shown to perform better on narrower domains

Page Rank – Discussion

- \+ Offline computation independently from the query – this allows for computing the PageRank for billions and more Web pages!
- \+ PageRank emphasises popular pages
- \+ Good results for home page search
- +- Page Rank prefers entry pages of Web sites
- – Other methods have shown to perform better on narrower domains
- – Possible manipulation with link farms
  - This is due to the independence of query and content
  - It's possible to just buy many (e.g. 10,000) incoming links to get a better PageRank – they do not need to be about the same topic!

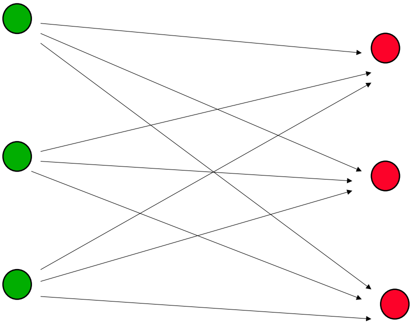
Kleinberg's HITS Algorithm

[Kleinberg, 1998]

Determining hubs and authorities of a certain topic

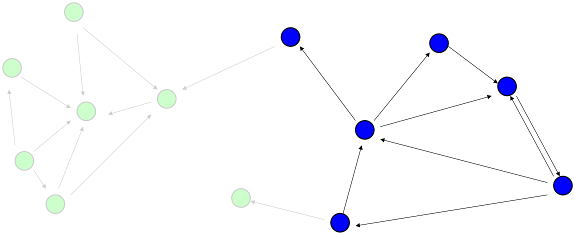
Hub Links to good authorities

Authority Has many links from good hubs



HITS: 2-step Algorithm

1. Compute the RSV for each Web site w.r.t. the query. Based on this, choose the subnet of relevant documents (**neighbourhood graph**)



2. Compute hub and authority values for each Web site in the neighbourhood graph

Hub and Authority Computation

Iterative Computation of authority a_p and hub value h_p

$$
\begin{aligned}
a_p &= \sum_{q \to p} h_q \\
h_q &= \sum_{q \to p} a_p
\end{aligned}
$$

a_p: Authority weight for node/page p
h_q: Hub weight for node/page q

and **normalisation criterium**

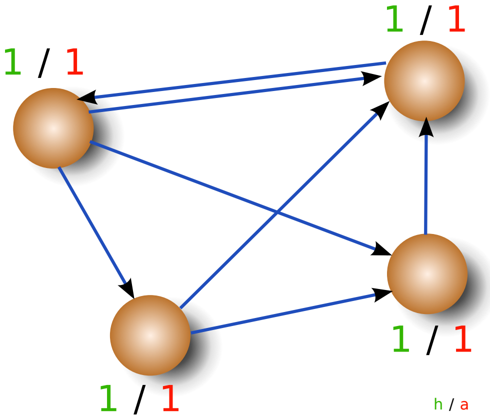
$$\sum_p (a_p)^2 = 1 \quad \text{and} \quad \sum_q (h_q)^2 = 1$$

Outline of the Iterative HITS Algorithm

1. Initialise each page with a hub and authority weight of 1
2. Compute hub and authority weights for each node
3. Normalise according to normalisation criterium by dividing each a_p by $\sqrt{\sum_p (a_p)^2}$ and each h_q by $\sqrt{\sum_q (h_q)^2}$
4. Goto 2, if convergence criterium not satisfied

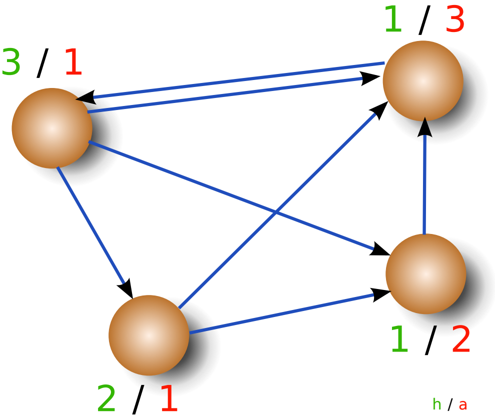
HITS Example

Start



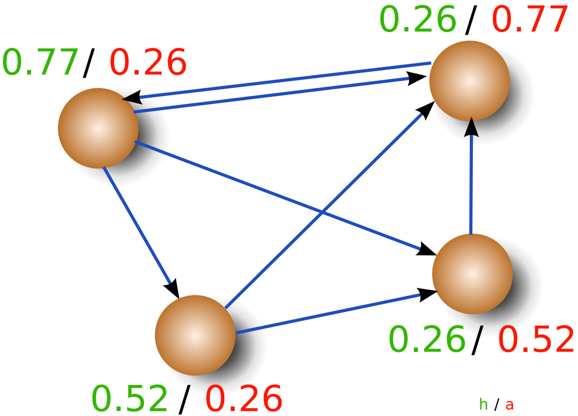
HITS Example

1st Iteration



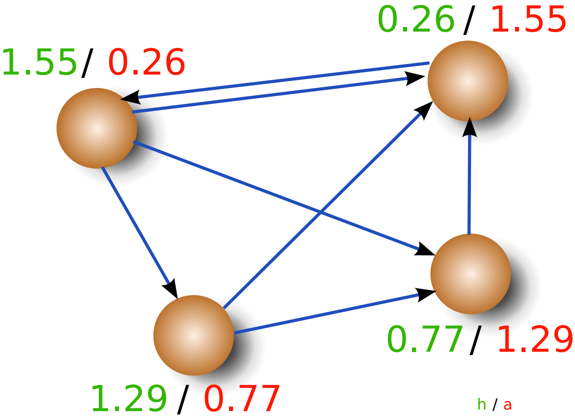
HITS Example

Normalisation



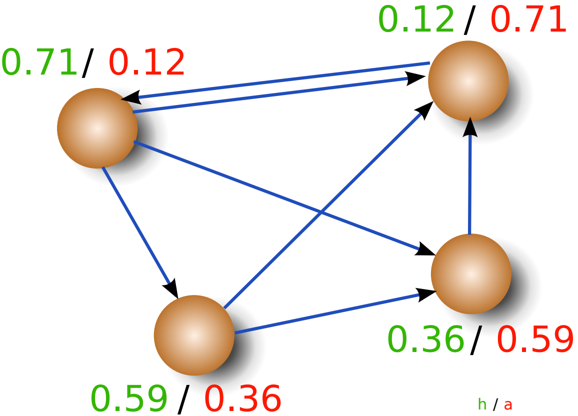
HITS Example

2nd Iteration



HITS Example

Normalisation



HITS – Discussion

+ Computation of distinct hub and authority values allow for more complex search strategies
  - Authority values are comparable to the PageRank and identify “important” documents $\rightarrow$ search for important and authoritative documents
  - Hubs usually are good surveys due to their links to very authoritative documents $\rightarrow$ search for good surveys

HITS – Discussion

- + Computation of distinct hub and authority values allow for more complex search strategies
  - Authority values are comparable to the PageRank and identify "important" documents $\rightarrow$ search for important and authoritative documents
  - Hubs usually are good surveys due to their links to very authoritative documents $\rightarrow$ search for good surveys
- + Hub and authority values are in line with the query and therefore on topic

HITS – Discussion

+ Computation of distinct hub and authority values allow for more complex search strategies
    - Authority values are comparable to the PageRank and identify “important” documents $\rightarrow$ search for important and authoritative documents
    - Hubs usually are good surveys due to their links to very authoritative documents $\rightarrow$ search for good surveys
+ Hub and authority values are in line with the query and therefore on topic
– Computation at query time, therefore longer response times

Conclusion

Conclusion

- Browsing and Searching in the World Wide Web
- The Anatomy of a Web Search Engine – Google
- The Web as a Graph: Link Analysis
  - Combination of RSV with link analysis
  - PageRank
  - HITS

References

References I

Brin, S. and Page, L. (1998).
The anatomy of a large-scale hypertextual web search engine.
In *Proceedings of the 7th International World-Wide Web Conference (WWW 1998)*, pages 107–117, Brisbane, Australia. Elsevier Science.

Broder, A. (2002).
A taxonomy of web search.
SIGIR Forum, 36:3–10.

Broder, A., Kumar, R., Maghoui, F., Raghavan, P., Rajagopalan, S., Stata, R., Tomkins, A., and Wiener, J. (2000).
Graph structure in the web: Experiments and models.
In *Proceedings of the Ninth Conference on World Wide Web*, pages 309–320, Amsterdam, The Netherlands. ACM.

References II

Kleinberg, J. (1998).
Authoritative sources in a hyperlinked environment.
In *Proceedings of the 9th Annual ACM-SIAM Symposium on Discrete Algorithms*, pages 668–677.

Page, L., Brin, S., Motwani, R., and Winograd, T. (1998).
The PageRank Citation Ranking: Bringing Order to the Web.
Technical report, Stanford Digital Library Technologies Project.

Rose, D. E. and Levinson, D. (2004).
Understanding User Goals in Web Search.
In *Proceedings of the 13th International Conference on World Wide Web*, pages 13–19, New York, NY, USA. ACM.